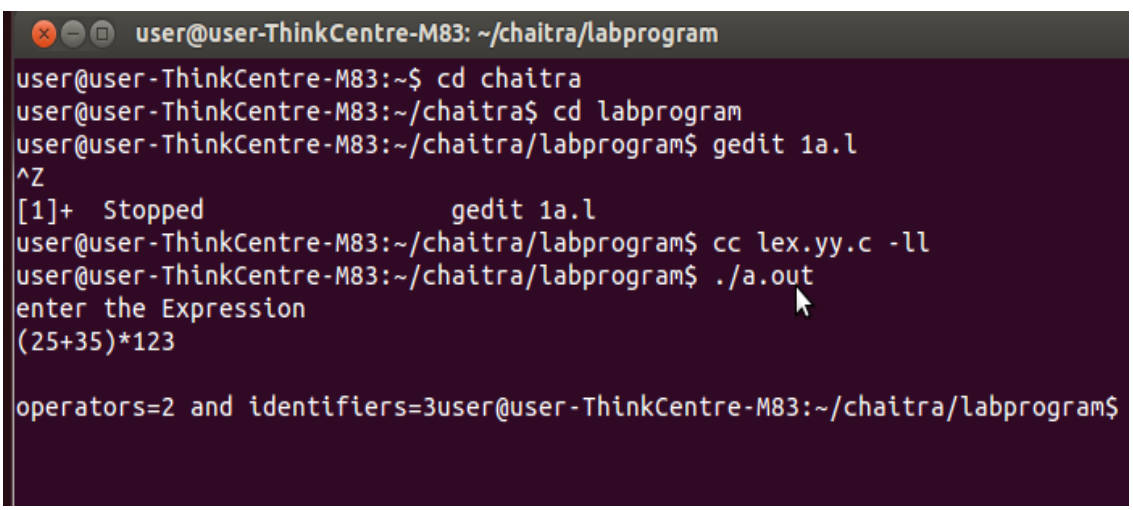
1. a. Write a LEX program to recognize valid arithmetic expression. Identifiers in the expression could be only integers and operators could be + and *. Count the identifiers & operators present and print them separately.

```
%{
        #include<stdio.h>
        int op=0,id=0,ob=0,cb=0,flag=1;
%}
%%
[+*]            op++;
[0-9]+          id++;
"("             ob++;
")"             cb++;
[-/%a-zA-Z] flag=0;
%%
void main()
{
        printf("enter the Expression \n");
        yylex();
        if(flag==0 || ob!=cb)
                printf("invalid expression");
        else
                printf("operators=%d and identifiers=%d\n",op,id);
}
```

Output:



```
user@user-ThinkCentre-M83: ~/chaitra/labprogram
user@user-ThinkCentre-M83:~$ cd chaitra
user@user-ThinkCentre-M83:~/chaitra$ cd labprogram
user@user-ThinkCentre-M83:~/chaitra/labprogram$ gedit 1a.l
^Z
[1]+  Stopped                 gedit 1a.l
user@user-ThinkCentre-M83:~/chaitra/labprogram$ cc lex.yy.c -ll
user@user-ThinkCentre-M83:~/chaitra/labprogram$ ./a.out
enter the Expression
(123/12)

invalid expressionuser@user-ThinkCentre-M83:~/chaitra/labprogram$
```

1. **b. Write YACC program to evaluate arithmetic expression involving operators: +, -, *, and /**

LEX PART:

```
%{

        #include "y.tab.h"
       extern int yylval;
%}

%%
[0-9]+      {yylval=atoi(yytext);return number;}
.            {return yytext[0];}
%%
```

YACC PART:

```
%{
        #include<stdio.h>
       #include<stdlib.h>
        int res=0;
%}

%token number
%left '+' '-'
%left '*' '/'

%%
stmt:expr {res=$$;}
      ;
expr:expr '+' expr {$$=$1+$3;}
      |expr '-' expr {$$=$1-$3;}
      |expr '*' expr {$$=$1*$3;}
      |expr '/' expr {if($3==0)
                        exit(0);
                        else $$=$1/$3;}
       |'(' expr ')'   {$$=$2;}
       |number
       ;
%%
```

```
void main()
{
        printf(" enter expr\n");
        yyparse();
        printf("Result is =%d \n",res);
        exit(0);
}


yyerror()
{
        printf("invalid\n");
        exit(0);
}
```

Output:

```
user@user-ThinkCentre-M83:~$ cd chaitra
user@user-ThinkCentre-M83:~/chaitra$ cd labprogram
user@user-ThinkCentre-M83:~/chaitra/labprogram$ lex 1b.l
user@user-ThinkCentre-M83:~/chaitra/labprogram$ yacc -d 1b.y
user@user-ThinkCentre-M83:~/chaitra/labprogram$ cc lex.yy.c y.tab.c -ll
user@user-ThinkCentre-M83:~/chaitra/labprogram$ ./a.out
 enter expr
(4-2)*12

Result is =24
user@user-ThinkCentre-M83:~/chaitra/labprogram$ ./a.out
 enter expr
(2^3)+2
invalid
user@user-ThinkCentre-M83:~/chaitra/labprogram$ ./a.out
 enter expr
4-2*12

Result is =-20
user@user-ThinkCentre-M83:~/chaitra/labprogram$
```

OR

```
%{
        #include<stdio.h>
        #include<ctype.h>
        #define YYSTYPE double
%}
%token num
%left '+' '-'
%left '*' '/'
%%
lines:lines exp '\n' {printf("\n value =%g \n\n",$2); return 0;}
      |lines '\n'
      |
      ;
exp:exp '+' exp  {$$=$1+$3;}
    |exp '-' exp  {$$=$1-$3;}
    |exp '*' exp  {$$=$1*$3;}
    |exp '/' exp  {if($3==0.0) printf("divide by zero error");
  else
  $$=$1/$3;
  }
   |('exp')'    {$$=$2;}
   |'-'exp      {-$2;}
   |num {$$=$1;}
   ;
%%
int yylex()
{
        char c;
        c=getchar();
        if((c=='.')||(isdigit(c)))
        {
                ungetc(c,stdin);
                scanf("%lf",&yylval);
                return num;
        }
        return c;
}
int main()
{
        printf("enter the expression\n");
```

```
        yyparse();
        return 0;
}

int yyerror()
{
        printf("error\n");
        return 0;
}
```

Output:

```
user@user-ThinkCentre-M83:~$ gedit 1b.y
^Z
[1]+  Stopped                 gedit 1b.y
user@user-ThinkCentre-M83:~$ yacc 1b.y
user@user-ThinkCentre-M83:~$ cc y.tab.c
user@user-ThinkCentre-M83:~$ ./a.out
enter the expression
3-2*9

 value =-15

user@user-ThinkCentre-M83:~$ ./a.out
enter the expression
3*5-
error
user@user-ThinkCentre-M83:~$
```

2. **Develop, Implement and Execute a program using YACC tool to recognize all strings ending with b preceded by n a's using the grammar an b (note: input n value)**

```
%{
        #include<stdio.h>
        #include<stdlib.h>
        int n,count=0;
%}

%token a b

%%
start:s'\n'  {if (count==n)
                 printf("valid\n");
             else
                 printf("Invalid \n");}

s:a s       {count++;}
 |b
 ;
%%

void main()
{
         printf("enter the value of n \n");
         scanf("%d",&n);
         getchar();
         printf("enter the string \n");
         yyparse();
}

yylex()
{
          char c;
          c=getchar();
          if(c=='a') return a;
          if (c=='b') return b;
          return c;
}
```

```
int yyerror()
{
        printf("error \n");
         exit(0);
}
```

Output:

```
user@user-ThinkCentre-M83: ~/chaitra/labprogram
user@user-ThinkCentre-M83:~/chaitra/labprogram$ gedit 2yacc.y
^Z
[2]+  Stopped                 gedit 2yacc.y
user@user-ThinkCentre-M83:~/chaitra/labprogram$ yacc -d 2yacc.y
user@user-ThinkCentre-M83:~/chaitra/labprogram$ cc y.tab.c -ll
user@user-ThinkCentre-M83:~/chaitra/labprogram$ ./a.out
enter the value of n
3
enter the string
aaab
valid
```

```
user@user-ThinkCentre-M83: ~/chaitra/labprogram
user@user-ThinkCentre-M83:~$ cd chaitra
user@user-ThinkCentre-M83:~/chaitra$ cd labprogram
user@user-ThinkCentre-M83:~/chaitra/labprogram$ yacc -d 2yacc.y
user@user-ThinkCentre-M83:~/chaitra/labprogram$ cc y.tab.c -ll
user@user-ThinkCentre-M83:~/chaitra/labprogram$ ./a.out
enter the value of n
2
enter the string
aa
error
user@user-ThinkCentre-M83:~/chaitra/labprogram$ yacc -d 2yacc.y
user@user-ThinkCentre-M83:~/chaitra/labprogram$ cc y.tab.c -ll
user@user-ThinkCentre-M83:~/chaitra/labprogram$ ./a.out
enter the value of n
2
enter the string
aaab
Invalid
```

3. **Design, develop and implement YACC/C program to construct Predictive / LL(1) Parsing Table for the grammar rules: A→aBa , B→bB | ε. Use this table to parse the sentence: abba$**

```
#include<stdio.h>
//#include<conio.h>
#include<stdlib.h>
#include<string.h>
void main()
{
	char M[3][4][10]={"NT/T","a","b","$","A","aBa","Error","Error","B","@","bB","Error"};
	char input[20],stack[20],temp[20];
	int i,j,k,T,NT,top=-1;
	//clrscr();
	printf("LL-1 predective parsing table is \n");
	for(i=0;i<3;i++)
	{
		for(j=0;j<4;j++)
			printf("%s\t",M[i][j]);
		printf("\n");
	}
	printf("Enter the String to be parsed\n");
	scanf("%s",input);
	input[strlen(input)]='$';
	stack[++top]='$';
	stack[++top]='A';
	i=j=k=0;
	while(stack[top]!='$' || input[i]!='$')
	{
		if(stack[top]>64 && stack[top]<91)
		{
			switch(stack[top])
			{
				case 'A': NT=1; break;
				case 'B': NT=2; break;
			}
			//if stack top is non-terminal
			switch(input[i])
			{
				case 'a': T=1; break;
```

```
                        case 'b': T=2; break;
                        case '$': T=3; break;
                        default : printf("\n Error while parsing the string \n");
                        exit(0);
                }
                strcpy(temp,M[NT][T]);
                if(strcmp(temp,"Error")==0)
                {
                        printf("\n Error while parsing the string \n");
                        exit(0);
                }
                else if(strcmp(temp,"@")==0)
                        top--;
                else
                {
                        top--;
                        for(k=strlen(temp)-1;k>=0;k--)
                                stack[++top]=temp[k];
                }
        }
        else //if stack top is terminal
        {
                if(stack[top]==input[i])
                {
                        top--;
                        i++;
                }
                else
                {
                        printf("\n Error while parsing the string \n");
                        exit(0);
                }
        }
    }
    printf("String belongs to given grammar \n");
    //getch();
}
```

Output:

```
user@user-ThinkCentre-M83:~/chaitra/labprogram$ gedit 3.c
^Z
[1]+  Stopped                 gedit 3.c
user@user-ThinkCentre-M83:~/chaitra/labprogram$ cc 3.c
user@user-ThinkCentre-M83:~/chaitra/labprogram$ ./a.out
LL-1 predective parsing table is
NT/T    a       b       $
A       aBa     Error   Error
B       @       bB      Error
Enter the String to be parsed
abba
String belongs to given grammar
user@user-ThinkCentre-M83:~/chaitra/labprogram$ ./a.out
LL-1 predective parsing table is
NT/T    a       b       $
A       aBa     Error   Error
B       @       bB      Error
Enter the String to be parsed
abbbb

 Error while parsing the string
user@user-ThinkCentre-M83:~/chaitra/labprogram$
```

4. Design, develop and implement YACC/C program to demonstrate *Shift Reduce* Parsing technique for the grammar rules: **E→E+T | T, T→T*F | F, F→(E) | id** and parse the sentence: ***id + id * id.***

```
#include<stdio.h>
#include<string.h>
int k=0,z=0,i=0,j=0,c=0;
char a[16],stk[15];
void Reduce();
void main()
{
        puts("GRAMMAR is \n E->E+E \n E->E*E \n E->(E) \n E->id");
       puts("enter input string ");
        gets(a);
       c=strlen(a);

       puts("stack \t input \t action");

       for(k=0,i=0; j<c; k++,i++,j++)
       {
               if(a[j]=='i' && a[j+1]=='d')
              {
                      stk[i]=a[j];
                      stk[i+1]=a[j+1];
                      stk[i+2]='\0';
                     a[j]=' '; a[j+1]=' ';
                      printf("\n$%s\t%s$\t SHIFT id",stk,a);
                      Reduce();
               }
               else
              {
                     stk[i]=a[j];
                     stk[i+1]='\0';
                     a[j]=' ';
                      printf("\n$%s\t%s$\t SHIFT symbols",stk,a);
                       Reduce();
              }
       }

}
```

```
void Reduce()
{
        for(z=0; z<c; z++)
                if(stk[z]=='i' && stk[z+1]=='d')
                {
                        stk[z]='E';
                         stk[z+1]='\0';
                         printf("\n$%s\t%s$\t REDUCE TO E",stk,a);
                         j++;
                }
        for(z=0; z<c; z++)
                 if(stk[z]=='E' && stk[z+1]=='+' && stk[z+2]=='E')
                {
                         stk[z]='E';
                         stk[z+1]='\0';
                        stk[z+2]='\0';
                        printf("\n$%s\t%s$\t REDUCE TO E",stk,a);
                         i=i-2;
                }
        for(z=0; z<c; z++)
                if(stk[z]=='E' && stk[z+1]=='*' && stk[z+2]=='E')
                {
                        stk[z]='E';
                         stk[z+1]='\0';
                         stk[z+1]='\0';
                        printf("\n$%s\t%s$\t REDUCE TO E",stk,a);
                         i=i-2;
                 }

       for(z=0; z<c; z++)
                 if(stk[z]='(' && stk[z+1]=='E' && stk[z+2]==')')
                 {
                         stk[z]='E';
                         stk[z+1]='\0';
                        stk[z+1]='\0';
                         printf("\n$%s\t%s$\t REDUCE TO E",stk,a);
                         i=i-2;
                }
  }
```

Output:

```
user@user-ThinkCentre-M83:~$ cd chaitra
user@user-ThinkCentre-M83:~/chaitra$ cd labprogram
user@user-ThinkCentre-M83:~/chaitra/labprogram$ gedit 4.c
^Z
[1]+  Stopped                 gedit 4.c
user@user-ThinkCentre-M83:~/chaitra/labprogram$ cc 4.c
user@user-ThinkCentre-M83:~/chaitra/labprogram$ ./a.out
GRAMMAR is
 E->E+E
 E->E*E
 E->(E)
 E->id
enter input string
id+id*id
stack    input   action

$id       +id*id$        SHIFT id
$E        +id*id$        REDUCE TO E
$E+        id*id$        SHIFT symbols
$E+id        *id$        SHIFT id
$E+E         *id$        REDUCE TO E
$E           *id$        REDUCE TO E
$E*           id$        SHIFT symbols
$E*id           $        SHIFT id
$E*E            $        REDUCE TO E
$E              $        REDUCE TO Euser@user-ThinkCentre-M83:~/chaitra/labprogr
user@user-ThinkCentre-M83:~/chaitra/labprogram$
```

5. **Design, develop and implement a C/Java program to generate the machine code using Triples for the statement A = -B * (C +D) whose intermediate code in three-address form:** ***T1 = -B***

   T2 = C + D

   T3 = T1 + T2 A = T3

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<ctype.h>
char op[2],arg1[5],arg2[5],result[5],eq[2];
void main()
{
	FILE *fp1;
	fp1=fopen("input3.txt","r");
	while(!feof(fp1))
	{
		fscanf(fp1,"%s%s%s%s%s",result,eq,arg1,op,arg2);
		if(strcmp(arg1,"T3")==0)
		{
			printf("\nMOV R0,%s",arg1);
			printf("\n MOV %s,R0\n",result);
		}
		else
		if(strcmp(op,"+")==0)
		{
			printf("\n MOV R0,%s",arg1);
			printf("\n ADD R0,%s",arg2);
			printf("\n MOV %s,R0",result);
		}
		else
		if(strcmp(op,"*")==0)
		{
			printf("\n MOV R0,%s",arg1);
			printf("\n MUL R0,%s",arg2);
			printf("\n MOV %s,R0",result);
		}
		else
		if(strcmp(op,"-")==0)
		{
```

```
                    printf("\n MOV R0,-%s",arg2);
                    printf("\n MOV %s,R0",result);
            }
        }
fclose(fp1);
}
```

Output:

```
user@user-ThinkCentre-M83:~$ cd chaitra
user@user-ThinkCentre-M83:~/chaitra$ cd labprogram
user@user-ThinkCentre-M83:~/chaitra/labprogram$ gedit 05.c
^Z
[1]+  Stopped                 gedit 05.c
user@user-ThinkCentre-M83:~/chaitra/labprogram$ cc 05.c
user@user-ThinkCentre-M83:~/chaitra/labprogram$ ./a.out

 MOV R0,-B
 MOV T1,R0
 MOV R0,C
 ADD R0,D
 MOV T2,R0
 MOV R0,T1
 MUL R0,T2
 MOV T3,R0
MOV R0,T3
 MOV A,R0
user@user-ThinkCentre-M83:~/chaitra/labprogram$
```

6. a) Write a LEX program to eliminate comment lines in a C program and copy the resulting program into a separate file.

```
%{
        #include<stdio.h>
        #include<stdlib.h>
I       nt l=0;
%}

%%
"/*"[^*]*"*/"        l++;
"//".*               l++;
%%
int main()
{
        yyin=fopen("new.c","r");
        yyout=fopen("new1.c","w");

        yylex();

        fclose(yyin);
        fclose(yyout);
        printf("no of comment lines are =%d \n",l);
}
```

Output:

```
user@user-ThinkCentre-M83:~/chaitra/labprogram$ lex 6a.l
user@user-ThinkCentre-M83:~/chaitra/labprogram$ cc lex.yy.c -ll
user@user-ThinkCentre-M83:~/chaitra/labprogram$ ./a.out
no of comment lines are =3
user@user-ThinkCentre-M83:~/chaitra/labprogram$ cat new.c
/*program to add two numbers*/
int a,b;//global declaration
void main()
{
int sum;
sum=a+b;
/*print the numbers*/
  printf("sum=%d",sum);
}
user@user-ThinkCentre-M83:~/chaitra/labprogram$ cat new1.c

int a,b;
void main()
{
int sum;
sum=a+b;

  printf("sum=%d",sum);
}
user@user-ThinkCentre-M83:~/chaitra/labprogram$
```

6. b). Write YACC program to recognize valid *identifier, operators and keywords* in the given text (*C program*) file.

LEX PART:

```
%{

        #include "y.tab.h"
        extern int yylval;
%}

%%
int|char|float|if|else|printf        {printf("keyword   is - %s \n",yytext); return key;}
[a-zA-Z][a-zA-Z0-9]*                 {printf("identifier is- %s \n",yytext); return id;}
[*+-/(),<>=]                         {printf("operator  is - %s \n",yytext); return op;}
[0-9]+                               {printf("number  is- %s \n",yytext); return num;}
.                                     ;
\n                                    ;
"\"".*"\""                           {printf("string  is- %s \n",yytext);}
%%
```

YACC PART:

```
%{
        #include<stdio.h>
        #include<stdlib.h>
         int k=0,i=0,n=0,opr=0;
         extern FILE *yyin;
%}

%token num id op key

%%
s: key          {k++;}
 | op           {opr++;}
 | num          {n++;}
 | id           {i++;}
 | id  s        {i++;}
 | num s        {n++;}
 | op  s        {opr++;}
 | key s         {k++;}
 ;
%%
```

```
void main()
{
        yyin=fopen("input.c","r");
        while(!feof(yyin))
        yyparse();

       printf(" \n key=%d and op=%d and id=%d and num=%d",k,opr,i,n);
}

yyerror()
{
        printf("invalid\n");
       exit(0);
}
```

Output:

```
user@user-ThinkCentre-M83:~/chaitra/labprogram$ lex 6b.l
user@user-ThinkCentre-M83:~/chaitra/labprogram$ yacc -d 6b.y
user@user-ThinkCentre-M83:~/chaitra/labprogram$ cc lex.yy.c y.tab.c -ll
user@user-ThinkCentre-M83:~/chaitra/labprogram$ ./a.out
identifier is- void
identifier is- main
operator  is - (
operator  is - )
keyword   is - int
identifier is- a
operator  is - =
number  is- 10
operator  is - ,
identifier is- b
operator  is - =
number  is- 20
keyword   is - if
operator  is - (
identifier is- a
operator  is - >
identifier is- b
operator  is - )
keyword   is - printf
operator  is - (
string  is- "a is greater"
operator  is - )

 key=3 and op=10 and id=6 and num=2user@user-ThinkCentre-M83:~/chaitra/labprogra
user@user-ThinkCentre-M83:~/chaitra/labprogram$ cat input.c
void main()
{
        int a=10,b=20;
        if(a>b)
        printf("a is greater");
}
```

7. **Design, develop and implement a C/C++/Java program to simulate the working of Shortest remaining time and Round Robin (RR) scheduling algorithms. Experiment with different quantum sizes for RR algorithm**

```
#include<stdio.h>

int n,qtum,pid[10],a[10],b[10],tt[10],avg[10];
int flag[10],flag1[10],swt=0,stat=0,rbt[10],i_total_burst_time=0;
float f_avg_turn_arnd_time=0.0,f_avg_wait_time=0.0;

rr()
{
	int ch;
	int st[10];
	int i,count=0,temp,sq=0;
	printf("enter the no. of processors\n");
	scanf("%d",&n);
	printf("enter the burst time\n");
	for(i=0;i<n;i++)
	{
		pid[i]=i+1;
		scanf("%d",&b[i]);
	}
	printf("enter the quantum:");
	scanf("%d",&qtum);
	for(i=0;i<n;i++)
	{
		st[i]=b[i];
	}
	while(1)
	{
		for(i=0;i<n;i++)
		{
			temp=qtum;
			if(st[i]==0)
			{
				count++;
				continue;
			}
			if(st[i]>qtum)
```

```
                st[i]=st[i]-qtum;
                else
                if(st[i]>=0)
                {
                        temp=st[i];
                        st[i]=0;
                }
                sq=sq+temp;
                tt[i]=sq;
        }
        if(n==count)
        break;
    }
    printf("\n process id burst time wait time turn around time \n");
    for(i=0;i<n;i++)
    {
        avg[i]=tt[i]-b[i];
        printf("%d\t\t%d\t%d\t%d",i+1,b[i],avg[i],tt[i]);
        printf("\n");
    }
    for(i=0;i<n;i++)
    {
        swt=swt+avg[i];
        stat=stat+tt[i];
    }
    f_avg_wait_time=(float)swt/n;
    f_avg_turn_arnd_time=(float)stat/n;
    printf("\n\n Average waiting time is %f\n Average turn around time is
%f\n",f_avg_wait_time,f_avg_turn_arnd_time);
    return 1;
}

int srtf()
{
    int ch;
    int a[10],b[10],x[10],i,j,smallest,count=0,time=0,n;
    double avg=0,tt=0,end;
    printf("enter the no. of processes:\n");
    scanf("%d",&n);
    printf("enter arrival time\n");
```

```
        for(i=0;i<n;i++)
                scanf("%d",&a[i]);
        printf("enter the burst time\n");
        for(i=0;i<n;i++)
                scanf("%d",&b[i]);
        for(i=0;i<n;i++)
                x[i]=b[i];
        b[9]=9999;
        for(time=0;count!=n;time++)
        {
                smallest=9;
                for(i=0;i<n;i++)
                {
                        if(a[i]<=time && b[i]<b[smallest] && b[i]>0)
                                smallest=i;
                }
                b[smallest]--;
                if(b[smallest]==0)
                {
                        count++;
                        end=time+1;
                        avg=avg+end-a[smallest]-x[smallest];
                        tt=tt+end-a[smallest];
                }
        }
        printf("\n Average waiting time=%lf\n",avg/n);
        printf("Average turn around time=%lf\n",tt/n);
        return 0;
}

int main()
{
        int i,ch;
        printf("1.SRTF\n");
        printf("2.RR\n");
        printf("3.exit\n");
        printf("enter the choice\n");
        scanf("%d",&ch);
        switch(ch)
        {
```

```
                case 1:srtf();
                        break;
                case 2:rr();
                        break;
                case 3:return 1;
        }
        return 1;
}
```

Output:

```
user@user-ThinkCentre-M83: ~/chai/labprogram
user@user-ThinkCentre-M83:~/chai/labprogram$ ./a.out
1.SRTF
2.RR
3.exit
enter the choice
1
enter the no. of processes:
5
enter arrival time
0
1
2
3
4
enter the burst time
6
5
3
4
2

 Average waiting time=5.600000
Average turn around time=9.600000
user@user-ThinkCentre-M83:~/chai/labprogram$
```

```
user@user-ThinkCentre-M83:~$ cd chai
user@user-ThinkCentre-M83:~/chai$ cd labprogram
user@user-ThinkCentre-M83:~/chai/labprogram$ cc rr.c
user@user-ThinkCentre-M83:~/chai/labprogram$ ./a.out
1.SRTF
2.RR
3.exit
enter the choice
2
enter the no. of processors
5
enter the burst time
6
7
5
4
3
enter the quantum:2




 process id burst time wait time turn around time
1               6       15      21
2               7       18      25
3               5       19      24
4               4       14      18
5               3       16      19


 Average waiting time is 16.400000
 Average turn around time is 21.400000
user@user-ThinkCentre-M83:~/chai/labprogram$
```

8. Design, develop and implement a C/C++/Java program to implement Banker's algorithm. Assume suitable input required to demonstrate the results.

```
#include<stdio.h>

struct process
{
        int allocation[6],max[6],need[6],finished, request[6];
 } p[10];

int avail[6], sseq[10], ss=0, check1=0, check2=0, n, pid, work[6];
int nor, nor_i;

void main()
 {
        int safeseq(void);
       int tj,ch,i=0,j=0,k,pid,ch1;

       printf("\n Enter number of processes : ");
       scanf("%d",&n);

        printf("\n Enter the Number of Resources : ");
        scanf("%d", &nor);

       printf("\n Enter the Available Resources : ");
        for(j=0;j<n;j++)
       {
               for(k=0; k<nor; k++)
               {
                       if(j==0)
                       {
                               printf("\n For Resource type %d : ",k);
                               scanf("%d", &avail[k]);
                        }

                        p[j].max[k]=0;
                       p[j].allocation[k]=0;
                       p[j].need[k]=0;
                        p[j].finished=0;
                        p[j].request[k]=0;
```

```
        }
 }

 printf("\n Enter Max resources for all processes\n");
 for(i=0;i<n;i++)
{
        for(j=0; j< nor; j++)
        {
                scanf("%d",&p[i].max[j]);
        }
}

printf("\n Enter Allocated resources for all processes\n");
for(i=0;i<n;i++)
 {
        for(j=0;j<nor;j++)
        {
                scanf("%d",&p[i].allocation[j]);

                if(p[i].allocation[j]>p[i].max[j])
                {
                        printf("\n Allocation should be less < or == max");
                        j--;
                }
                else
                p[i].need[j]=p[i].max[j]-p[i].allocation[j];
        }
 }
 if(safeseq()==1)
 {
        printf("\n The System is in Safe state\n ");
 }
 else
        printf("\n The System is Not in safe state\n ");
 printf("\n Need\n");
 for(i=0;i<n;i++)
{
        for(j=0;j<nor;j++)
        printf(" %d ",p[i].need[j]);
        printf("\n");
```

```
        }
}

int safeseq()
{
        int tk,tj,i,j,k;
        ss=0;
        for(j=0; j<nor; j++)
         work[j] = avail[j];
         for(j=0;j<n;j++)
        p[j].finished=0;
         for(tk=0; tk<nor; tk++)
         {
                for(j=0;j<n;j++)
                {
                        if(p[j].finished==0)
                        {
                                check1=0;
                                for(k=0; k<nor; k++)
                                        if(p[j].need[k]<=work[k])
                                                check1++;
                                if(check1== nor)
                                {
                                        for(k=0;k< nor;k++)
                                        {
                                                work[k]= work[k]+p[j]. allocation[k];
                                                p[j]. finished=1;
                                        }
                                        sseq[ss]=j;
                                        ss++;
                                }
                        }
                }
         }
        check2=0;
         for(i=0;i<n;i++)
                if(p[i].finished==1)
                        check2++;
        if(check2>=n)
        {
```

```
                printf("The Safe Sequence is\t :");
                for(tj=0;tj<n;tj++)
               printf("%d, ", sseq[tj]);
               return 1;
        }
        return 0;
}
```

Output:

```
user@user-ThinkCentre-M83: ~/chaitra/labprogram
user@user-ThinkCentre-M83:~$ cd chaitra
user@user-ThinkCentre-M83:~/chaitra$ cd labprogram
user@user-ThinkCentre-M83:~/chaitra/labprogram$ gedit bankers.c
user@user-ThinkCentre-M83:~/chaitra/labprogram$ cc bankers.c
user@user-ThinkCentre-M83:~/chaitra/labprogram$ ./a.out

 Enter number of processes : 5

 Enter the Number of Resources : 3

 Enter the Available Resources :
 For Resource type 0 : 3

 For Resource type 1 : 3

 For Resource type 2 : 2

 Enter Max resources for all processes
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
```

```
user@user-ThinkCentre-M83: ~/chaitra/labprogram

 Enter Max resources for all processes
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3

 Enter Allocated resources for all processes
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
The Safe Sequence is     :1, 3, 4, 0, 2,
 The System is in Safe state

 Need
 7  4  3
 1  2  2
 6  0  0
 0  1  1
 4  3  1
user@user-ThinkCentre-M83:~/chaitra/labprogram$
```

9. **Design, develop and implement a C/C++/Java program to implement page replacement algorithms LRU and FIFO. Assume suitable input required to demonstrate the results.**

LRU:

```
#include<stdio.h>
#include<stdlib.h>

char frm[10],s[20];
void main()
{
	int m=0,k,f=3,l,i=0,flag=0;
	printf("enter the frame size\n");
	scanf("%d",&f);
	printf("enter the characters(or)no of pages required\n");
	scanf("%d",&l);
	printf("enter the 10 characters\n",l);
	scanf("%s",s);
	printf("page frame res\n");
	for(i=0;i<l;i++)
	{
		flag=0;
		for(k=0;k<f;k++)
			if(s[i]==frm[k])
			{
				flag=1;
				break;
			}
		if(m<f && flag==0)
		{
			frm[m]=s[i];
			m++;
			printf("%c\t%s\t page fault\n",s[i],frm);
		}
		else if(flag!=1)
		{
			for(k=0;k<m-1;k++)
			frm[k]=frm[k+1];
			frm[k]=s[i];
```

```
                printf("%c\t%s\t page fault \n",s[i],frm);
            }
            else
            {
                for(k=k;k<m-1;k++)
                frm[k]=frm[k+1];
                frm[k]=s[i];
                printf("%c\t%s\t no page fault \n",s[i],frm);
            }
        }
}
```

FIFO:

```
#include<stdio.h>
#include<stdlib.h>

char frm[10],s[20];
void main()
{
        int m=0,k,f=3,l,i=0,flag=0;
        printf("enter the frame size\n");
        scanf("%d",&f);
        printf("enter the characters(or)no of pages required\n");
        scanf("%d",&l);
        printf("enter the 10 characters\n",l);
        scanf("%s",s);
        printf("page frame res\n");
        for(i=0;i<l;i++)
        {
            flag=0;
            for(k=0;k<f;k++)
                if(s[i]==frm[k])
                {
                    flag=1;
                    break;
                }
            if(m<f && flag==0)
            {
                frm[m]=s[i];
                m++;
```

```
                printf("%c\t%s\t page fault\n",s[i],frm);
            }
            else if(flag!=1)
            {
                for(k=0;k<m-1;k++)
                frm[k]=frm[k+1];
                frm[k]=s[i];
                printf("%c\t%s\t page fault \n",s[i],frm);
            }
            else
            {
                printf("%c\t%s\t no page fault \n",s[i],frm);
            }
        }
}
```

Output:

LRU:

```
user@user-ThinkCentre-M83: ~/chaitra/labprogram
user@user-ThinkCentre-M83:~/chaitra/labprogram$ gedit lru.c
user@user-ThinkCentre-M83:~/chaitra/labprogram$ cc lru.c
user@user-ThinkCentre-M83:~/chaitra/labprogram$ ./a.out
enter the frame size
3
enter the characters(or)no of pages required
10
enter the 10 characters
ababcbeheh
page frame res
a       a        page fault
b       ab       page fault
a       ba       no page fault
b       ab       no page fault
c       abc      page fault
b       acb      no page fault
e       cbe      page fault
h       beh      page fault
e       bhe      no page fault
h       beh      no page fault
user@user-ThinkCentre-M83:~/chaitra/labprogram$
```

FIFO:

```
user@user-ThinkCentre-M83: ~/chaitra/labprogram
user@user-ThinkCentre-M83:~/chaitra/labprogram$ gedit fifo.c
user@user-ThinkCentre-M83:~/chaitra/labprogram$ cc fifo.c
user@user-ThinkCentre-M83:~/chaitra/labprogram$ ./a.out
enter the frame size
3
enter the characters(or)no of pages required
10
enter the 10 characters
ababcbeheh
page frame res
a       a       page fault
b       ab      page fault
a       ab      no page fault
b       ab      no page fault
c       abc     page fault
b       abc     no page fault
e       bce     page fault
h       ceh     page fault
e       ceh     no page fault
h       ceh     no page fault
user@user-ThinkCentre-M83:~/chaitra/labprogram$ gedit fifo.c
user@user-ThinkCentre-M83:~/chaitra/labprogram$
```